Terminal Buddy: Technical Build Specification

Lead Architect: Fred **Core Language:** Rust 1.80+ (Stable) **Target Platforms:** Linux (X11/Wayland), macOS (AppKit/Metal), Windows (DirectX)

1. The Multi-Process Architecture

We do not use a monolithic design. We utilize a **Coupled Daemon Pattern** to ensure the UI thread is never blocked by heavy AI or File I/O operations.

Process A: `buddy-gui` (The Driver)

- **Crate:** `gpui` (Zed Industries)
- **Memory Model:** Immediate-mode UI logic.
- **Responsibility:** * Texture mapping for the terminal grid.
  - Low-latency event loop (Keyboard/Mouse/Voice triggers).
  - IPC Client (talking to the Daemon).
- **Threading:** Single main UI thread; background tasks delegated to `gpui::Task`.

Process B: `buddy-daemon` (The Sidecar)

- **Crate:** `tokio` (Async Runtime)
- **Responsibility:** * **PTY Management:** Spawning and controlling `nushell` or `bash` via `portable-pty`.
  - **The Black Box:** Implementing the `Memvid` writer loop.
  - **System Sensors:** Polling `sysinfo` and `notify-rust`.
  - **MCP Host:** Managing JSON-RPC connections to AI tools.
- **IPC:** `interprocess` crate using Unix Domain Sockets (Linux/macOS) or Named Pipes (Windows).

2. The "Hard Space" Memory Engine (Memvid)

The "Personal Log" is built using the **Memvid SDK**.

- **Mechanism:** Instead of a traditional database, the Daemon captures the terminal's internal buffer (the "Grid") and the system delta (CPU/Disk) every 100ms or on command completion.

- **Format:** `.mv2`. This is a binary stream where each frame contains:
  1. `TerminalState`: A compressed delta of the text grid.
  2. `SystemImpact`: A JSON blob of the disk/CPU changes.
  3. `Embedding`: A vector representation for semantic search.
- **Performance:** <5ms append time. The `.mv2` file acts as a **Write-Ahead Log (WAL)**, making it crash-safe.

3. The Control Plane (Neovim & Voice)

Neovim Embedding

- **Crate:** `nvim-rs`
- **Build:** We spawn `nvim --embed --headless`.
- **Logic:** The `buddy-gui` implements the Neovim UI protocol. Every character typed in "Edit Mode" is sent to the headless Neovim instance; the resulting grid updates are rendered by `gpui`. This gives the user full Vim power with zero overhead.

Voice Pipeline (The Aider Logic)

- **Crate:** `cpal` (Audio input) + `whisper-rs` (Bindings for whisper.cpp).
- **Flow:** 1. User holds `Right Ctrl` (Hotmapped in `buddy.toml`). 2. `buddy-gui` streams raw PCM audio to `buddy-daemon`. 3. `buddy-daemon` runs local transcription. 4. Transcribed text is injected into the PTY stdin or the Neovim buffer.

4. System Introspection (The Teachable Moment)

To provide the "Cause & Effect" analysis, we implement a **State-Diffing Engine** in Rust.

```
pub struct Introspector {
    last_snap: SystemSnapshot,
}

impl Introspector {
    pub fn diff(&mut self, current: SystemSnapshot) -> Delta {
        let d = Delta {
            disk_usage: current.disk_bytes - self.last_snap.disk_bytes,
            proc_spawned: current.pids.difference(&self.last_snap.pids),
        };
        self.last_snap = current;
        d
    }
```

```
}
```

- **Visualization:** This `Delta` is written to the Memvid file and read by the **Canvas Dashboard** (React) to show the user exactly how their computer changed.

5. Development Directives for Phase 1

1. **Repo Structure:** Cargo Workspace with `gui/` and `daemon/` members.
2. **UI Foundation:** Initialize `gpui::App`. Render a basic `TextElement`.
3. **PTY Proof:** Spawn a shell and pipe a single `ls` command output into the `gpui` window.
4. **IPC Proof:** Send a "Hello" message from GUI to Daemon and log it to a `.mv2` file.